Data-Driven Methods for Engineers
(MECH0107)
- Coursework 1 -

Spring Pendulum: The Theory of Hybrid Systems

2025 – 2026

Contents

1 Introduction and Problem Statement **1**

2 Methodology **1**
2.1 Motion Tracking (Image Processing) 1
2.2 Time Frequency Analysis (Gabor Transform) 3
2.3 Dimensionality Reduction (SVD) 3

3 Results **4**
3.1 Tracked Displacement Data 4
3.2 Spectrograms and Frequency Evolution 4
3.3 SVD Mode Decomposition 5

4 Discussion **6**
4.1 Physical Interpretation of Modes 6
4.2 Mode Coupling and Nonlinearity 6
4.3 Simplified Analytical Model 7
4.4 Sources of Error and Improvements 7

5 Conclusion **8**

6 Appendix **9**
6.1 Part 1 9
6.2 Part 2 13
6.3 Part 3 16
6.4 Supplementary Figures 18
6.5 Additional Figures 22

1 Introduction and Problem Statement

The elastic pendulum, also known as the spring pendulum, is a classical example of a nonlinear hybrid dynamical system (Breitenberger and Mueller, 1981). It consists of a mass suspended from a spring, free to oscillate both vertically (along the spring axis) and horizontally (in the plane of swing). The elastic pendulum couples two fundamentally different restoring mechanisms: the elastic spring force, governed by Hooke's law ($F = -kx$), and the gravitational restoring force that drives pendulum-like swinging. At certain energy levels the system exhibits near-periodic energy exchange between the two modes, and at others it can behave chaotically (Lynch, 2002). The interaction becomes particularly pronounced when the natural frequency of the spring mode is approximately twice that of the pendulum mode, a condition known as autoparametric resonance (Nayfeh and Mook, 1979).

In this coursework, a physical spring pendulum system was recorded using three cameras placed at different orientations around the mass. The mass was released from an off-centre position, simultaneously exciting both the vertical spring oscillation and the horizontal pendulum swing. The video data from each camera were provided as MATLAB `.mat` files (`cam1.mat`, `cam2.mat`, `cam3.mat`), each containing a sequence of RGB video frames. Since no frame rate metadata was available, all frequency quantities are expressed in cycles per frame rather than in Hz throughout this report.

The analysis is structured around three main objectives. The first is to extract the position of the oscillating mass from each camera's video using image processing techniques, producing displacement time series for subsequent analysis. The second is to characterise how the frequency content of the motion evolves over time, which requires time-frequency analysis rather than a conventional Fourier transform, since the system's dynamics are non-stationary. The third is to reduce the dimensionality of the multi-camera dataset and identify the dominant modes of motion, linking each mode to its physical origin in the spring-pendulum dynamics.

To achieve these objectives, the following workflow was adopted. In Part 1, image processing techniques, including phase correlation for video stabilisation (Kuglin and Hines, 1975), brightness-based segmentation, binary thresholding, and connected component analysis, were applied to each camera's video to extract the centroid trajectory of the mass. In Part 2, the Gabor transform (Gabor, 1946) was applied to the displacement signals to produce spectrograms, revealing how the dominant frequencies change over time. Three different Gaussian window widths were compared to illustrate the trade-off between time and frequency resolution. In Part 3, singular value decomposition (Kutz, 2013) was applied to the combined multi-camera displacement matrix to extract the principal modes of motion, assess how many modes are needed to represent the system, and interpret each mode in terms of the underlying physics.

2 Methodology

This section describes the methods used to extract, analyse, and interpret the motion data from the three camera recordings.

2.1 Motion Tracking (Image Processing)

The 2D position of the mass was extracted from each camera's video using the six step pipeline illustrated in Figure 1. This approach was chosen because phase correlation is robust to intensity changes and does not require feature detection, the brightness channel $\max(R, G, B)$ reliably isolates the bright mass against the darker background without colour-space tuning, and connected component labelling handles varying blob shapes at low computational cost. Together these steps provide accurate, noise-tolerant tracking with minimal manual parameter selection.

Video stabilisation: Small frame to-frame camera shifts were removed by phase correlation (Kuglin and Hines, 1975). Each frame was converted to grayscale using the NTSC luminance weights ($I = 0.299R + 0.587G + 0.114B$) (Gonzalez and Woods, 2018) and its translational shift estimated from the peak of the inverse FFT of the normalised cross-power spectrum:

$$C = \frac{\hat{f}_1 \cdot \overline{\hat{f}_2}}{|\hat{f}_1 \cdot \overline{\hat{f}_2}|} \tag{1}$$

where $\hat{f}$ denotes the 2D FFT and $\bar{\cdot}$ the complex conjugate. Cumulative shifts were applied to align all frames to frame 0.

ROI, brightness, and thresholding: Each stabilised frame was cropped to a region of interest (ROI) around the mass to exclude background clutter (Gonzalez and Woods, 2018). The brightness was defined as the per-pixel channel maximum:

$$B(x, y) = \max\big(R(x, y),\, G(x, y),\, B(x, y)\big) \tag{2}$$

A fixed threshold θ then produced a binary mask:

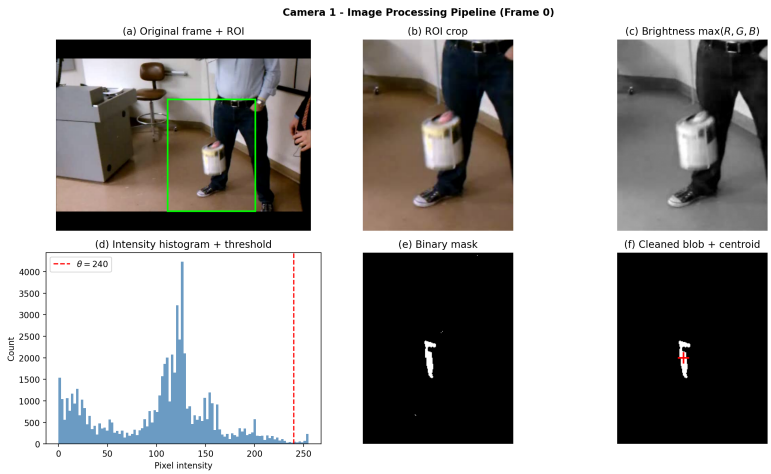
$$M(x, y) = \begin{cases} 1 & B(x, y) > \theta \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

Values ($\theta = 240$, 245, 200 for Cameras 1-3) were chosen from the intensity histogram of each camera.

Morphological cleanup and centroid: Connected component labelling (Gonzalez and Woods, 2018) isolated distinct blobs. Those with area outside [20, 8000] px were discarded, and the blob nearest the previous centroid was retained. The mass position was taken as the centre-of-mass:

$$(c_x,\, c_y) = \left(\frac{\sum_{(x,y)\in\mathcal{B}} x}{|\mathcal{B}|},\, \frac{\sum_{(x,y)\in\mathcal{B}} y}{|\mathcal{B}|} \right) \tag{4}$$

Frames with no valid detection were filled by linear interpolation, and the mean position was subtracted to produce centred displacement signals $\Delta x(t)$ and $\Delta y(t)$. Camera 3 required an additional $90° + 24°$ rotation before processing. The extra 24° was chosen so that the base of the paint can appeared horizontal and parallel with the ground in the corrected frames. Visual inspection of the raw footage also revealed that Camera 2 began recording approximately five frames earlier than the other two cameras. These leading frames were trimmed prior to processing to synchronise the three recordings.



(a) Original frame + ROI
(b) ROI crop
(c) Brightness $\max(R, G, B)$
(d) Histogram + threshold $\theta = 240$
(e) Binary mask
(f) Cleaned blob + centroid

Figure 1: Image processing pipeline for Camera 1, Frame 0.

2.2 Time Frequency Analysis (Gabor Transform)

The spring pendulum is non stationary: the pendulum swing damps out while the spring oscillation persists, so frequencies evolve with time. A global Fourier transform cannot capture this because it averages over the entire recording. Instead, the Gabor transform (Gabor, 1946) was applied, which localises the spectrum in time by windowing the signal before transforming.

Window function: For each sliding time position τ, a Gaussian window is centred on the signal $S(t)$:

$$g_\tau(t) = \exp\left(-\frac{(t-\tau)^2}{a^2}\right) \tag{5}$$

The windowed signal $S(t) \cdot g_\tau(t)$ is then Fourier transformed to give the local frequency content at time τ. Repeating across all τ builds the full spectrogram $\mathcal{G}(\tau, \omega)$.

Window width and resolution trade off: The width a controls the balance between time and frequency resolution, as required by the Heisenberg uncertainty principle (Kutz, 2013): a narrow window (a small) gives good temporal localisation but blurs frequency. A wide window gives sharp frequency peaks but poor time resolution. Three widths were compared, $a = 5$, 15, 40 frames, to illustrate this trade-off. Since no frame-rate metadata was available, all frequencies are reported in cycles per frame throughout.

2.3 Dimensionality Reduction (SVD)

The six displacement signals (x and y from three cameras) were stacked into a data matrix and decomposed using singular value decomposition (SVD) to identify the dominant modes of motion shared across cameras. SVD was chosen over PCA because the two are mathematically equivalent on mean-centred data (Kutz, 2013), since PCA diagonalises the covariance matrix, which is exactly $U\Sigma^2U^\top$ from SVD, but SVD is more numerically stable for small matrices and directly provides both the spatial modes (U) and temporal coefficients (V) in a single decomposition, making physical interpretation straightforward. This eigenvalue-based decomposition approach has been successfully applied in other domains such as face recognition (Turk and Pentland, 1991), demonstrating its generality as a dimensionality reduction tool.

Data matrix and mean centering: The matrix $X \in \mathbb{R}^{6\times n}$ was formed by stacking the displacement signals as rows (cam1 x, cam1 y, cam2 x, cam2 y, cam3 x, cam3 y). Row means were subtracted to give the centred matrix $B = X - \bar{X}$.

SVD decomposition: Following Kutz (2013), SVD was applied to the scaled matrix:

$$\frac{B}{\sqrt{n-1}} = U\,\Sigma\,V^\top \tag{6}$$

where the columns of U are the spatial modes (left singular vectors), $\Sigma = \mathrm{diag}(\sigma_1, \ldots, \sigma_6)$ contains the singular values, and V holds the temporal coefficients. The scaling by $\sqrt{n-1}$ makes the singular values equivalent to the square roots of eigenvalues of the sample covariance matrix, so that:

$$\lambda_k = \frac{\sigma_k^2}{\sum_j \sigma_j^2} \tag{7}$$

gives the fraction of total variance explained by mode k. The temporal projection of each mode was computed as $Y = U^\top B$, and low-rank reconstructions $\hat{X}_r = U_r\,\Sigma_r\,V_r^\top\sqrt{n-1} + \bar{X}$ were used to assess how many modes are needed to represent the data.

3 Results

3.1 Tracked Displacement Data

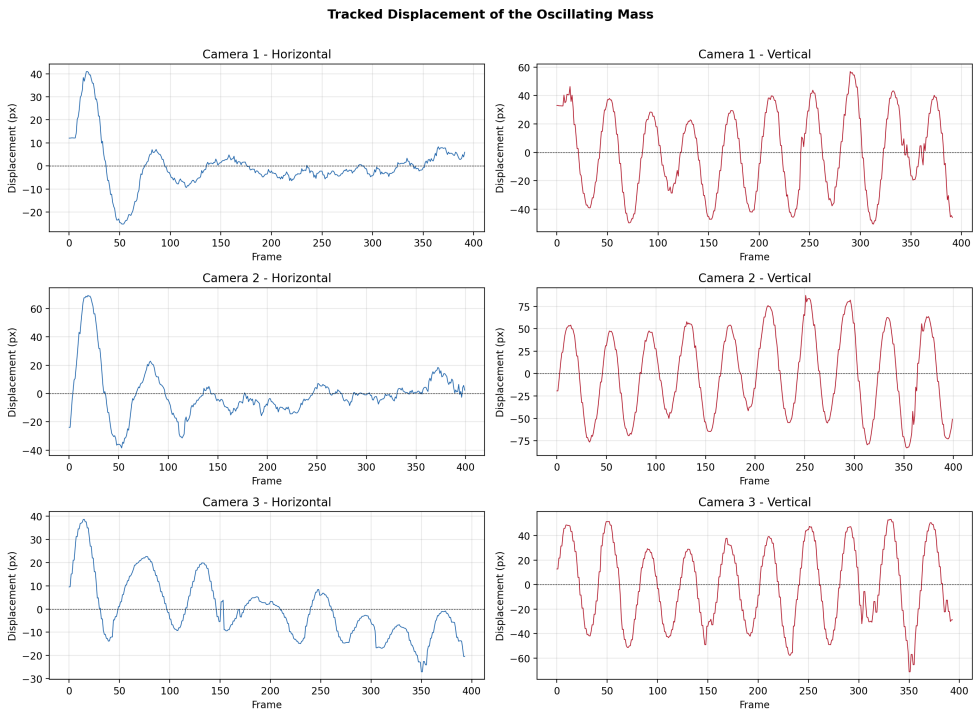


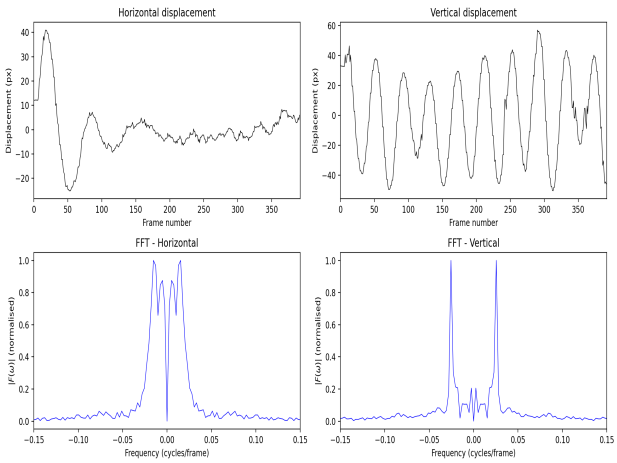
Figure 2: Mean centred displacement time series from all three cameras.

Figure 2 shows the mean centred displacement signals extracted from each camera. In Camera 1 (side view), the horizontal component shows a clear oscillation whose amplitude decays steadily over the recording, consistent with a damped pendulum swing. The vertical component oscillates at a visibly higher frequency and remains largely sustained throughout, indicating the persistent spring-mode vibration. Cameras 2 and 3 capture different projections of the same underlying motion, and all three cameras show this characteristic two-frequency behaviour, with the lower-frequency component damping out faster than the higher-frequency one. Some irregularity is visible in the signals because the paint can also rotates and sways slightly out of plane during oscillation, rather than swinging as an ideal point mass. The 2D trajectories in Figure 6 (Appendix) further illustrate these deviations from a simple back-and-forth path.

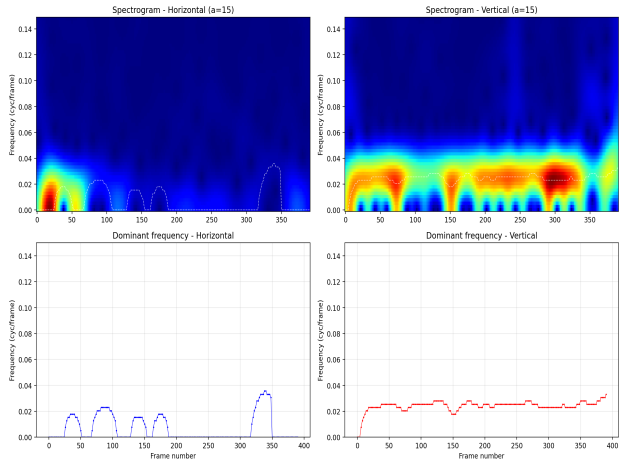
3.2 Spectrograms and Frequency Evolution

The global FFT of the Camera 1 signals (Figure 3a) shows two distinct frequency peaks. The vertical displacement has a dominant peak at approximately twice the frequency of the horizontal peak, consistent with the 2:1 ratio expected at autoparametric resonance (Nayfeh and Mook, 1979; Lynch, 2002). The spectrograms (Figure 3b) reveal how each component evolves in time: the horizontal frequency band is strong early on but decays and becomes noisy in the second half of the recording as the pendulum damps out, while the vertical frequency band remains coherent throughout. The effect of changing the Gabor window width on resolution is shown in Figures 7 and 8 (Appendix). The white dashed line tracking the instantaneous dominant frequency confirms this pattern, a relatively stable peak for the spring mode, and an increasingly scattered dominant frequency for the pendulum mode as its amplitude falls below the noise floor.

However, the spectrograms analyse each component independently and cannot identify shared patterns across cameras. This motivates the SVD in Part 3, which combines all six signals to extract the dominant modes and quantify the variance each explains.



(a) Displacement signals (top row) and global FFT magnitude spectra (bottom row) for Camera 1, showing two distinct frequency peaks.



(b) Spectrograms ($a = 15$ frames) with dominant frequency overlay (white dashed), and dominant frequency tracks over time.

Figure 3: Global FFT and time frequency analysis for Camera 1. The vertical (spring) component is sustained. The horizontal (pendulum) component damps out.

3.3 SVD Mode Decomposition

The singular value spectrum (Figure 4, left) shows a steep drop after the first mode, with Modes 1, 2, and 3 accounting for 80.9%, 10.0%, and 4.6% of total variance respectively. As shown in Figure 5, three modes together capture 95.5% of the variance, exceeding the conventional 95% threshold. A fourth mode brings the cumulative total to 98.2%. The fact that three modes are needed, rather than the two expected for a linear two-degree-of-freedom system, is indicative of nonlinear coupling.

The temporal projection of each mode (Figure 4, right) reveals distinct characters. Mode 1 oscillates at a nearly constant frequency throughout the recording with little amplitude decay, consistent with the sustained spring oscillation. Mode 2 oscillates at roughly half the frequency of Mode 1 and its amplitude drops to approximately a quarter of its initial value by the second half of the recording, consistent with the damped pendulum swing. Mode 3 remains sustained but captures a smaller fraction of the total variance.

The spatial structure of each mode (Figure 5, right) confirms these physical interpretations. Mode 1 has its largest weights on the vertical channels of all three cameras, consistent with spring-mode (vertical) dominance. Mode 2 is weighted most heavily on the horizontal channels, linking it to the pendulum swing. Mode 3 shows a mixed structure with opposite signs on Camera 2 and Camera 3 vertical channels, indicating that it captures a perspective-dependent component arising from the different camera viewing angles.

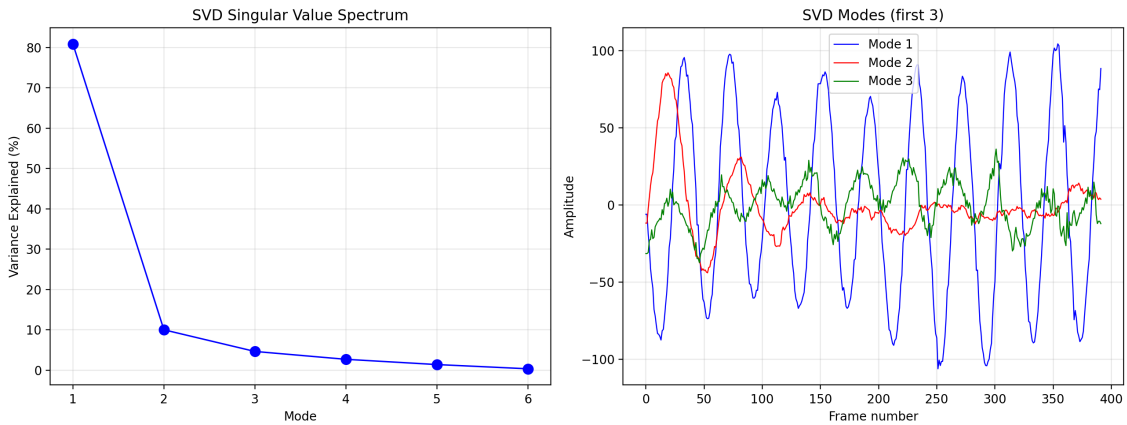
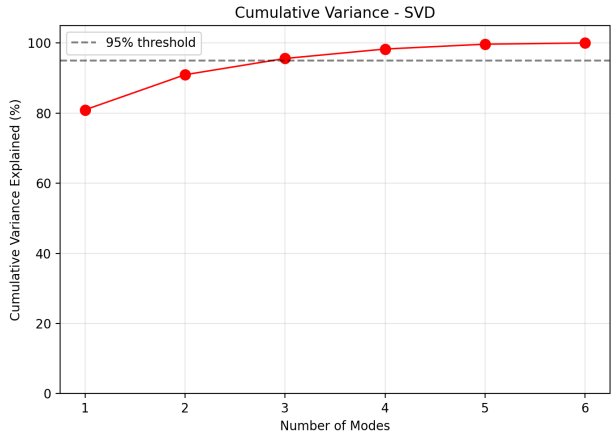
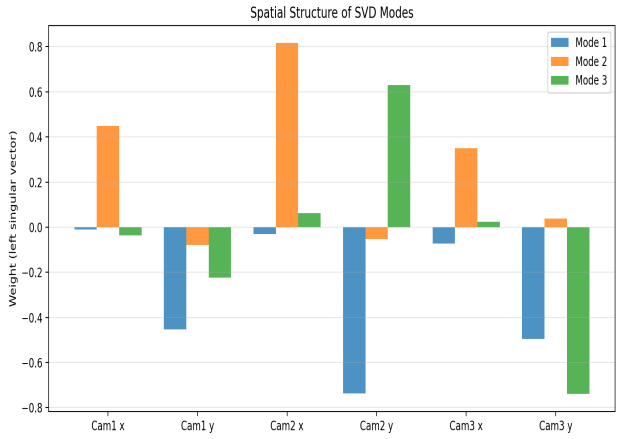


Figure 4: Left: variance explained by each SVD mode. Right: temporal projections of the first three modes. Mode 1 (blue) is sustained. Mode 2 (red) damps out.



(a) Cumulative variance. Three modes reach 95.5%. four modes reach 98.2%.



(b) Spatial structure (left singular vectors $\mathbf{U}$). Mode 1 is vertical dominant. Mode 2 is horizontal dominant.

Figure 5: Cumulative variance and spatial mode structure from the SVD decomposition.

Low rank reconstruction (Figure 9, Appendix) confirms these assignments: a rank 1 approximation captures only the spring oscillation, while adding the second mode recovers the pendulum swing and the third captures the residual perspective component.

4 Discussion

4.1 Physical Interpretation of Modes

The SVD decomposition cleanly separates the two physical degrees of freedom of the elastic pendulum (Breitenberger and Mueller, 1981). Mode 1, which accounts for 80.9% of the total variance, oscillates at a sustained, nearly constant amplitude throughout the recording and is weighted most heavily on the vertical channels of all three cameras (Figure 5b). This is consistent with the spring mode oscillation: the elastic restoring force $F = -k\Delta l$ drives vertical bouncing that loses energy slowly because the spring stores and releases it efficiently with relatively little aerodynamic drag. The dominance of this mode reflects the fact that vertical motion is the largest amplitude component of the system's dynamics.

Mode 2 (10.0% of variance) oscillates at roughly half the frequency of Mode 1 and its amplitude decays strongly over the recording, dropping to approximately a quarter of its initial value by the second half. Its spatial weights concentrate on the horizontal channels of all three cameras, identifying it as the pendulum swing. The faster damping makes physical sense, as the pendulum mode sweeps the mass through a larger arc for a given energy, increasing air resistance losses, and the gravitational restoring force is weaker than the spring force for small displacements.

Mode 3 (4.6%) is less straightforward to interpret. Its spatial structure shows opposite signs on the vertical channels of Camera 2 and Camera 3, while the horizontal weights are near zero. Because the three cameras are positioned at different heights and angles around the pendulum, each one captures a slightly different 2D projection of the same 3D motion. When the mass moves in depth (toward or away from a camera), perspective causes an apparent vertical shift whose direction depends on whether the camera sits above or below the mass's mean position. Mode 3 therefore captures the component of motion on which the cameras disagree, primarily the perspective dependent projection of depth and out of plane displacement, rather than representing a distinct physical degree of freedom.

4.2 Mode Coupling and Nonlinearity

A purely linear two degree of freedom system would require exactly two modes to fully describe the motion. The fact that three modes are needed to reach 95.5% of the variance is evidence that

the elastic pendulum exhibits nonlinear coupling between its modes. This is consistent with theory: the equations of motion for the spring-pendulum contain terms like $\dot{\theta}^2\ell$ and $\ddot{\theta}\Delta\ell$ that mix the two coordinates, so the system is not separable into independent oscillators (Nayfeh and Mook, 1979).

Further evidence of coupling appears in the spectrograms (Figure 3). The observed frequency ratio of approximately 2:1 between the vertical and horizontal components places the system near the autoparametric resonance condition (Lynch, 2002), where energy exchange between modes is most efficient. This is visible in the displacement data as a slow amplitude modulation on the vertical signal, where the spring oscillation periodically gains and loses energy as the pendulum swings. If the two modes were truly independent, each would simply decay monotonically.

The spectrograms also show that the dominant frequencies are not perfectly constant over time. The horizontal component's frequency becomes increasingly scattered as its amplitude drops, partly due to the signal falling into the noise floor, but also because the effective pendulum length changes as the spring stretches and compresses, slightly shifting the natural frequency.

4.3 Simplified Analytical Model

For small oscillations, the elastic pendulum can be modelled as two coupled degrees of freedom (Lynch, 2002): the spring extension $\Delta\ell$ about the static equilibrium length $\ell_0 = L_0 + mg/k$, and the swing angle θ. The linearised natural frequencies are:

$$\omega_{\text{spring}} = \sqrt{\frac{k}{m}}, \qquad \omega_{\text{pendulum}} = \sqrt{\frac{g}{\ell_0}} \tag{8}$$

The 2:1 frequency ratio observed in the data ($\omega_{\text{spring}} \approx 2\,\omega_{\text{pendulum}}$) implies that $k/m \approx 4g/\ell_0$, which is precisely the autoparametric resonance condition derived in the theory of parametrically coupled oscillators (Nayfeh and Mook, 1979). Since no frame rate was available, we cannot extract absolute values of k or ℓ_0, but the ratio itself confirms the system operates near this resonance.

Discrepancies between this linear model and the observed behaviour, such as the amplitude-dependent frequency drift and the need for more than two SVD modes, arise from the geometric nonlinearity that the small angle approximation neglects. The full equations of motion contain products of θ, $\dot{\theta}$, and $\Delta\ell$ that become significant at the amplitudes present early in the recording. Additionally, damping (which the conservative model ignores) causes the pendulum mode to decay faster than the spring mode, breaking the symmetry of the energy exchange.

4.4 Sources of Error and Improvements

Several sources of error affect the reliability of the results. The most significant is the absence of frame rate metadata, which prevents converting frequencies from cycles per frame to physical units (Hz) and makes it impossible to extract absolute physical parameters like spring stiffness or pendulum length. All frequency analysis in this report is therefore relative rather than absolute.

The image based tracking introduces noise from several sources. The brightness thresholding approach is sensitive to lighting variations and reflections, which can shift the detected centroid by a few pixels between frames. The phase correlation stabilisation (Kuglin and Hines, 1975) corrects for rigid camera motion but cannot account for lens distortion or vibrations at sub pixel scales. The connected component labelling occasionally picks up spurious bright spots, particularly for Camera 3 where the contrast is lower and the threshold had to be set to 200 instead of 240-245.

Each camera provides only a 2D projection of the 3D motion, so the displacement signals mix the true degrees of freedom according to each camera's viewing angle. This is partly why Mode 3 in the SVD has a mixed spatial structure with opposing vertical weights across cameras, as it captures a perspective-dependent component that appears differently across the three views rather than representing a distinct physical mode. Using calibrated stereo cameras would allow full 3D reconstruction and cleaner mode separation (Hartley and Zisserman, 2003).

Potential improvements include: (i) recording the frame rate and using a physical scale marker to convert pixel displacements to SI units. (ii) Using a retro reflective marker on the mass for more robust tracking. (iii) Employing at least two calibrated cameras for stereo triangulation, and (iv) increasing the frame rate to improve temporal resolution in the spectrograms and reduce aliasing risk near the Nyquist limit.

5 Conclusion

This report analysed the dynamics of an elastic pendulum using video data from three cameras, applying image processing, time frequency analysis, and singular value decomposition to extract, characterise, and interpret the system's motion.

The image processing pipeline successfully tracked the mass across all three camera views, producing displacement time series that clearly show two distinct oscillatory components: a sustained, higher frequency spring mode and a decaying, lower frequency pendulum swing. The Gabor transform confirmed this two frequency structure and revealed how the pendulum component damps out over time while the spring mode persists, consistent with the expected physics of the system. The observed 2:1 frequency ratio places the system near the autoparametric resonance condition, where energy exchange between modes is most efficient.

SVD decomposition of the combined multi camera data identified three dominant modes accounting for 95.5% of the total variance. Mode 1 (80.9%) corresponds to the sustained spring oscillation and Mode 2 (10.0%) to the damped pendulum swing, together capturing the two physical degrees of freedom. Mode 3 (4.6%) captures a residual perspective dependent component arising from the different camera viewing angles. The fact that three modes are needed, rather than the two expected for a linear two degree of freedom system, provides evidence of nonlinear coupling and confirms that multi camera projection effects introduce additional variance that cannot be explained by the two physical modes alone.

Overall, the combination of Gabor analysis and SVD proved effective for this problem: the spectrograms captured the time varying frequency content that a global FFT would miss, while SVD separated the spatial modes that single camera analysis cannot distinguish. The main limitations stem from the lack of frame rate metadata and the 2D nature of the camera projections but potential solutions were addressed in part 4.4.

6 Appendix

The complete Python code for each part of the coursework is listed below. Part 1 handles motion tracking, Part 2 performs the Gabor time frequency analysis, Part 3 applies SVD for dimensionality reduction, and Supplementary Figures generates supplementary figures not produced by the main scripts.

6.1 Part 1

```python
"""
MECH0107 Coursework
Part 1

Extracts the position of an oscillating mass from video frames
captured by three cameras using image processing techniques.

Methodology:
    1. Video stabilisation via phase correlation (FFT-based)
    2. ROI cropping
    3. Brightness channel: max(R, G, B)
    4. Binary thresholding
    5. Morphological cleanup (connected component labelling)
    6. Centre-of-mass computation
"""

import numpy as np                          # numerical arrays and math
import scipy.io as sio                      # load .mat files
from scipy.ndimage import label, rotate     # blob labelling + image rotation
from scipy.ndimage import shift as ndi_shift  # sub-pixel frame shifting
import matplotlib.pyplot as plt             # plotting
import os                                   # file existence checks

# Configuration

VID_KEYS = {1: 'vidFrames1_4', 2: 'vidFrames2_4', 3: 'vidFrames3_4'}  # .mat variable names
    per camera
DATA_PATH = 'cam{}.mat'  # filename template for camera data

# Region of interest per camera: (y_min, y_max, x_min, x_max)
ROIS = {
    1: (150, 430, 280, 500),   # cam 1 crop bounds
    2: (80,  410, 180, 480),   # cam 2 crop bounds
    3: (200, 520, 129, 349),   # cam 3 crop bounds (after rotation)
}

# Brightness thresholds: chosen from the intensity histogram of each camera
THRESHOLDS = {1: 240, 2: 245, 3: 200}

MIN_BLOB_SIZE = 20    # ignore blobs smaller than this (noise)
MAX_BLOB_SIZE = 8000  # ignore blobs larger than this (background)


# Helper functions

def rotate_cam3_video(vid):
    """Rotate Camera 3 frames: 90 deg CW then 24 deg CW so the can base sits horizontal."""
    vid = np.rot90(vid, k=-1, axes=(0, 1))       # 90 deg clockwise
    for i in range(vid.shape[3]):                 # loop over each frame
        vid[:, :, :, i] = rotate(vid[:, :, :, i], angle=-24,
                                 reshape=False, order=1)  # extra 24 deg CW correction
    return vid


def stabilize_video(vid):
    """
    Stabilise a video using phase correlation.

    Phase correlation estimates the translational shift between frames
    using the normalised cross-power spectrum of their 2D FFTs. Each
    frame is aligned to frame 0 by accumulating frame-to-frame shifts.
    """
    nframes = vid.shape[3]            # total number of frames
    H, W = vid.shape[0], vid.shape[1] # frame height and width
```

```
    stabilised = vid.copy()                     # work on a copy so we keep the original
    shifts = np.zeros((nframes, 2))             # cumulative (dy, dx) for each frame

    # grayscale of first frame as initial reference
    prev_gray = (0.2989 * stabilised[:, :, 0, 0] +
                 0.5870 * stabilised[:, :, 1, 0] +
                 0.1140 * stabilised[:, :, 2, 0])  # standard luma weights

    for i in range(1, nframes):                      # start from frame 1
        curr = stabilised[:, :, :, i].astype(np.float64)  # current frame as float
        curr_gray = 0.2989 * curr[:, :, 0] + 0.5870 * curr[:, :, 1] + 0.1140 * curr[:, :, 2]
            # grayscale

        # cross-power spectrum
        F1 = np.fft.fft2(prev_gray)                 # FFT of previous frame
        F2 = np.fft.fft2(curr_gray)                 # FFT of current frame
        cross_power = F1 * np.conj(F2)              # element-wise cross-power
        mag = np.abs(cross_power)                   # magnitude for normalisation
        mag[mag == 0] = 1                           # avoid divide-by-zero
        correlation = np.real(np.fft.ifft2(cross_power / mag))  # inverse FFT gives
            correlation surface

        # peak location gives the shift
        peak = np.unravel_index(np.argmax(correlation), correlation.shape)  # find peak coords
        dy, dx = peak[0], peak[1]                   # raw shift values
        if dy > H // 2: dy -= H                     # wrap-around correction for negative
            shifts
        if dx > W // 2: dx -= W                     # same for horizontal

        shifts[i] = shifts[i - 1] + [dy, dx]        # accumulate shifts relative to frame 0

        # apply cumulative shift to align with frame 0
        for c in range(3):                           # loop R, G, B channels
            stabilised[:, :, c, i] = ndi_shift(
                vid[:, :, c, i].astype(np.float64),
                shift=-shifts[i], order=1, mode='constant', cval=0  # shift back by
                    accumulated offset
            ).astype(vid.dtype)

        prev_gray = (0.2989 * stabilised[:, :, 0, i] +
                     0.5870 * stabilised[:, :, 1, i] +
                     0.1140 * stabilised[:, :, 2, i])  # update reference to stabilised frame

    return stabilised, shifts


def rgb_to_brightness(rgb_image):
    """Brightness = max(R, G, B) per pixel. Isolates the brightest spots."""
    return np.max(rgb_image.astype(np.float64), axis=2)  # take max across colour channels


def apply_threshold(image, threshold):
    """Binary threshold: pixels above threshold -> 1, else -> 0."""
    return (image > threshold).astype(np.float64)  # simple binary mask


def morphological_cleanup(binary_image, min_size, max_size):
    """
    Connected component labelling followed by area filtering.
    Removes blobs smaller than min_size or larger than max_size.
    Returns the cleaned mask and a list of (cx, cy, area) for valid blobs.
    """
    labelled, num_features = label(binary_image)   # label connected regions
    cleaned = np.zeros_like(binary_image)           # output mask starts empty
    blob_info = []                                  # store centroid + area of valid blobs

    rows, cols = binary_image.shape                 # image dimensions
    yy, xx = np.meshgrid(np.arange(rows), np.arange(cols), indexing='ij')  # coordinate grids
        for centroid calc

    for k in range(1, num_features + 1):            # loop over each labelled blob
        mask = (labelled == k)                      # boolean mask for this blob
        area = int(np.sum(mask))                    # number of pixels in blob
        if area < min_size or area > max_size:      # skip if too small or too large
            continue
```

```
        cx = float(np.sum(xx * mask)) / area        # x centroid (weighted mean)
        cy = float(np.sum(yy * mask)) / area        # y centroid
        cleaned[mask] = 1                           # add blob to cleaned mask
        blob_info.append((cx, cy, area))            # record blob info

    return cleaned, blob_info


def select_best_blob(blob_info, prev_cx, prev_cy):
    """
    Pick the best blob: closest to previous position if available,
    otherwise the largest one.
    """
    if len(blob_info) == 0:                         # no blobs found
        return prev_cx, prev_cy                     # keep previous position
    if len(blob_info) == 1:                         # only one blob, use it
        return blob_info[0][0], blob_info[0][1]
    if prev_cx is not None:                         # multiple blobs and we have history
        best = min(blob_info, key=lambda b: (b[0]-prev_cx)**2 + (b[1]-prev_cy)**2)  # nearest
            to last known pos
    else:
        best = max(blob_info, key=lambda b: b[2])   # no history, pick the biggest blob
    return best[0], best[1]


# Main tracking pipeline for one camera

def track_camera(cam_id):
    """
    Full tracking pipeline for a single camera:
      stabilise -> crop ROI -> brightness -> threshold -> cleanup -> centroid
    """
    filepath = DATA_PATH.format(cam_id)             # build filename
    data = sio.loadmat(filepath)                    # load .mat file
    vid = data[VID_KEYS[cam_id]]                    # extract video array

    # Camera 2 starts ~5 frames early so trim to synchronise with other cameras
    if cam_id == 2:
        vid = vid[:, :, :, 5:]                      # drop first 5 frames

    if cam_id == 3:
        vid = rotate_cam3_video(vid)                # fix camera 3 orientation

    vid, _ = stabilize_video(vid)                   # stabilise all frames to frame 0
    nframes = vid.shape[3]                          # number of frames after trimming

    y1, y2, x1, x2 = ROIS[cam_id]                  # unpack ROI bounds
    threshold = THRESHOLDS[cam_id]                  # brightness threshold for this camera

    x_pos = np.full(nframes, np.nan)                # horizontal position array (fill NaN)
    y_pos = np.full(nframes, np.nan)                # vertical position array
    prev_cx, prev_cy = None, None                   # no previous position yet

    for i in range(nframes):                        # process each frame
        rgb_frame = vid[:, :, :, i].astype(np.float64)  # get frame as float
        roi = rgb_frame[y1:y2, x1:x2, :]           # crop to region of interest

        bright = rgb_to_brightness(roi)             # max(R,G,B) brightness
        binary = apply_threshold(bright, threshold) # threshold to binary
        cleaned, blob_info = morphological_cleanup(binary, MIN_BLOB_SIZE, MAX_BLOB_SIZE)  #
            filter blobs
        cx, cy = select_best_blob(blob_info, prev_cx, prev_cy)  # pick best blob centroid

        if cx is not None:                          # valid detection
            x_pos[i] = cx + x1                      # convert ROI coords back to full frame
            y_pos[i] = cy + y1                      # same for y
            prev_cx, prev_cy = cx, cy               # update tracking history

    # interpolate through any NaN gaps
    valid = ~np.isnan(x_pos)                        # mask of frames with valid detections
    if np.sum(~valid) > 0 and np.sum(valid) >= 2:   # only interpolate if there are gaps and
        enough good points
        idx = np.arange(nframes)                    # frame indices
        x_pos = np.interp(idx, idx[valid], x_pos[valid])  # linear interp over gaps
        y_pos = np.interp(idx, idx[valid], y_pos[valid])  # same for y
```

```python
    return x_pos, y_pos


# Main execution

if __name__ == '__main__':
    results = {}                                        # dict to hold results per camera
    for cam in [1, 2, 3]:                               # process all three cameras
        if not os.path.exists(DATA_PATH.format(cam)):  # check data file exists
            continue
        x, y = track_camera(cam)                        # run tracking pipeline
        results[cam] = {
            'x_raw': x, 'y_raw': y,                    # raw pixel positions
            'dx': x - np.nanmean(x),              # mean-centred horizontal displacement
            'dy': -(y - np.nanmean(y)),         # mean-centred vertical (negated because image y is
                flipped)
        }

    # save tracking data
    if results:                                         # only save if we got results
        np.savez('tracking_results.npz',
                 **{f'cam{c}_{ax}': results[c][f'd{ax}']
                    for c in results for ax in ['x', 'y']})  # save dx, dy for each camera

    # Figure 1: Displacement time series
    fig, axes = plt.subplots(3, 2, figsize=(14, 10))  # 3 cameras x 2 directions
    for i, cam in enumerate([1, 2, 3]):
        if cam not in results:
            continue
        frames = np.arange(len(results[cam]['dx']))     # frame numbers for x-axis
        axes[i, 0].plot(frames, results[cam]['dx'], color='#2166ac', linewidth=0.8)  #
            horizontal
        axes[i, 0].set_ylabel('Displacement (px)')
        axes[i, 0].set_title(f'Camera {cam} - Horizontal')
        axes[i, 0].set_xlabel('Frame')
        axes[i, 0].grid(True, alpha=0.3)
        axes[i, 0].axhline(0, color='k', lw=0.5, ls='--')  # zero reference line

        axes[i, 1].plot(frames, results[cam]['dy'], color='#b2182b', linewidth=0.8)  #
            vertical
        axes[i, 1].set_ylabel('Displacement (px)')
        axes[i, 1].set_title(f'Camera {cam} - Vertical')
        axes[i, 1].set_xlabel('Frame')
        axes[i, 1].grid(True, alpha=0.3)
        axes[i, 1].axhline(0, color='k', lw=0.5, ls='--')  # zero line

    plt.suptitle('Tracked Displacement of the Oscillating Mass',
                 fontsize=13, fontweight='bold', y=1.01)  # title
    plt.tight_layout()
    plt.savefig('fig_displacement.png', dpi=200, bbox_inches='tight')  # save figure
    plt.close()
```

6.2 Part 2

```python
"""
Part 2
Performs Gabor transform analysis on the displacement
data extracted in Part 1, using Camera 1.
"""

import numpy as np                          # numerical arrays and math
import matplotlib.pyplot as plt             # plotting

#Configuration

CAM = 1                                     # which camera to analyse

# Three window widths to demonstrate the time frequency trade off:
# narrow a = sharp time localisation, poor frequency resolution
# wide  a = sharp frequency resolution, poor time localisation
WIDTHS = [5, 15, 40]

TSLIDE_STEP = 2   # frames between consecutive window positions
MAX_FREQ = 0.15   # cycles/frame - upper limit for display


# Load data

data = np.load('tracking_results.npz')      # load Part 1 output
dx = data[f'cam{CAM}_x']                    # horizontal displacement
dy = data[f'cam{CAM}_y']                    # vertical displacement

# mean-subtract so the signal oscillates around zero
dx = dx - np.mean(dx)                       # remove DC offset from horizontal
dy = dy - np.mean(dy)                       # remove DC offset from vertical

n = len(dx)                                 # total number of frames
t = np.arange(n)                            # time axis (frame indices)

# frequency vector in cycles/frame
k = np.arange(-n//2, n//2) / n             # centred frequency axis for FFT plots



#Functions

def compute_fft(signal):
    """FFT magnitude spectrum, normalised to peak = 1."""
    mag = np.abs(np.fft.fftshift(np.fft.fft(signal)))  # FFT, shift zero-freq to centre, take
        magnitude
    return mag / np.max(mag)                            # normalise so peak = 1


def gabor_spectrogram(signal, width):
    """
    Slide a Gaussian window g(t) = exp(-(t-tau)^2 / a^2) across the signal,
    take the FFT at each position, and stack the magnitudes into a spectrogram.
    """
    tslide = np.arange(0, n, TSLIDE_STEP)              # window centre positions
    Sgt_spec = np.zeros((len(tslide), n))               # spectrogram matrix (time x freq)

    for j, tau in enumerate(tslide):                    # slide window across signal
        g = np.exp(-(t - tau)**2 / width**2)            # Gaussian window centred at tau
        Sgt = np.fft.fft(g * signal)                    # FFT of windowed signal
        mag = np.abs(np.fft.fftshift(Sgt))              # magnitude, shifted to centre
        Sgt_spec[j, :] = mag                            # store this time slice

    # normalise by global max so amplitude decay is visible across time
    if Sgt_spec.max() > 0:                              # avoid divide by zero
        Sgt_spec /= Sgt_spec.max()                      # scale to [0, 1]

    return tslide, Sgt_spec


# Figure 1: Signal + global FFT

mag_x = compute_fft(dx)                             # FFT of horizontal signal
mag_y = compute_fft(dy)                             # FFT of vertical signal
```

```
fig, axes = plt.subplots(2, 2, figsize=(13, 7))        # 2x2 grid: signals on top, FFTs below

axes[0, 0].plot(t, dx, 'k', linewidth=0.7)             # horizontal displacement vs time
axes[0, 0].set_title('Horizontal displacement')
axes[0, 0].set_xlabel('Frame number')
axes[0, 0].set_ylabel('Displacement (px)')
axes[0, 0].set_xlim([0, n])                            # full time range

axes[0, 1].plot(t, dy, 'k', linewidth=0.7)             # vertical displacement vs time
axes[0, 1].set_title('Vertical displacement')
axes[0, 1].set_xlabel('Frame number')
axes[0, 1].set_ylabel('Displacement (px)')
axes[0, 1].set_xlim([0, n])

axes[1, 0].plot(k, mag_x, 'b', linewidth=0.7)          # horizontal FFT spectrum
axes[1, 0].set_title('FFT - Horizontal')
axes[1, 0].set_xlabel('Frequency (cycles/frame)')
axes[1, 0].set_ylabel('$|F(\\omega)|$ (normalised)')
axes[1, 0].set_xlim([-MAX_FREQ, MAX_FREQ])             # zoom to relevant freq range

axes[1, 1].plot(k, mag_y, 'b', linewidth=0.7)          # vertical FFT spectrum
axes[1, 1].set_title('FFT - Vertical')
axes[1, 1].set_xlabel('Frequency (cycles/frame)')
axes[1, 1].set_ylabel('$|F(\\omega)|$ (normalised)')
axes[1, 1].set_xlim([-MAX_FREQ, MAX_FREQ])

plt.tight_layout()                                      # fix subplot spacing
plt.savefig('fig_part2_signal_fft.png', dpi=200, bbox_inches='tight')  # save figure
plt.close()


# Compute spectrograms with medium width for final analysis

w_mid = WIDTHS[1]                                       # use medium width (a=15)
freq_mask = (k >= 0) & (k <= MAX_FREQ)                  # only positive freqs up to MAX_FREQ
tslide, spec_x = gabor_spectrogram(dx, w_mid)           # horizontal spectrogram
_, spec_y = gabor_spectrogram(dy, w_mid)                # vertical spectrogram

freqs_pos = k[freq_mask]                                # positive frequency values only
spec_x_pos = spec_x[:, freq_mask]                       # crop spectrogram to positive freqs
spec_y_pos = spec_y[:, freq_mask]                       # same for vertical

dom_fx = freqs_pos[np.argmax(spec_x_pos, axis=1)]       # dominant freq at each time step (
    horizontal)
dom_fy = freqs_pos[np.argmax(spec_y_pos, axis=1)]       # dominant freq at each time step (
    vertical)

fig, axes = plt.subplots(2, 2, figsize=(14, 9))         # 2x2: spectrograms on top, freq
    tracks below

axes[0, 0].pcolormesh(tslide, freqs_pos, spec_x_pos.T, shading='auto', cmap='jet')  # horiz
    spectrogram
axes[0, 0].plot(tslide, dom_fx, 'w--', linewidth=0.8, alpha=0.7)  # overlay dominant freq line
axes[0, 0].set_title(f'Spectrogram - Horizontal (a={w_mid})')
axes[0, 0].set_ylabel('Frequency (cyc/frame)')

axes[0, 1].pcolormesh(tslide, freqs_pos, spec_y_pos.T, shading='auto', cmap='jet')  # vert
    spectrogram
axes[0, 1].plot(tslide, dom_fy, 'w--', linewidth=0.8, alpha=0.7)  # dominant freq overlay
axes[0, 1].set_title(f'Spectrogram - Vertical (a={w_mid})')
axes[0, 1].set_ylabel('Frequency (cyc/frame)')

axes[1, 0].plot(tslide, dom_fx, 'b.-', markersize=2, linewidth=0.6)  # horiz dominant freq
    over time
axes[1, 0].set_title('Dominant frequency - Horizontal')
axes[1, 0].set_xlabel('Frame number')
axes[1, 0].set_ylabel('Frequency (cyc/frame)')
axes[1, 0].set_ylim([0, MAX_FREQ])
axes[1, 0].grid(True, alpha=0.3)

axes[1, 1].plot(tslide, dom_fy, 'r.-', markersize=2, linewidth=0.6)  # vert dominant freq over
     time
axes[1, 1].set_title('Dominant frequency - Vertical')
axes[1, 1].set_xlabel('Frame number')
```

```
axes[1, 1].set_ylabel('Frequency (cyc/frame)')
axes[1, 1].set_ylim([0, MAX_FREQ])
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('fig_part2_dominant_freq.png', dpi=200, bbox_inches='tight')
plt.close()
```

6.3 Part 3

```
"""
Part 3
Applies SVD to the multi-camera displacement data from Part 1 to
identify the dominant modes of motion of the spring-pendulum system.
"""

import numpy as np                          # numerical arrays and math
import matplotlib.pyplot as plt             # plotting

#Load data
data = np.load('tracking_results.npz')      # load Part 1 tracking output

cam1_x = data['cam1_x'];  cam1_y = data['cam1_y']  # Camera 1 displacements
cam2_x = data['cam2_x'];  cam2_y = data['cam2_y']  # Camera 2 displacements
cam3_x = data['cam3_x'];  cam3_y = data['cam3_y']  # Camera 3 displacements

# truncate to the shortest camera recording
n = min(len(cam1_x), len(cam2_x), len(cam3_x))     # find shortest length
cam1_x, cam1_y = cam1_x[:n], cam1_y[:n]             # trim cam 1
cam2_x, cam2_y = cam2_x[:n], cam2_y[:n]             # trim cam 2
cam3_x, cam3_y = cam3_x[:n], cam3_y[:n]             # trim cam 3
t = np.arange(n)                                     # frame index array


#Step 1: Build data matrix: rows = measurements, cols = frames

X = np.vstack([cam1_x, cam1_y,
               cam2_x, cam2_y,
               cam3_x, cam3_y])                     # 6 x n data matrix (6 channels, n frames
                   )

m = X.shape[0]                                       # number of measurement channels (6)


# Step 2 & 3: Mean-subtract each row

mn = np.mean(X, axis=1, keepdims=True)# mean of each channel
B = X - mn # zero-mean data matrix


# Step 4: SVD
# Dividing by sqrt(n-1) so that S^2 gives eigenvalues of the covariance

U, S, Vt = np.linalg.svd(B / np.sqrt(n - 1), full_matrices=False)  # economy SVD

variance = S**2 / np.sum(S**2)# fraction of variance per mode
Y = U.T @ B  # project data onto SVD modes



# Figure 1: Singular value spectrum + first 3 modes

fig, axes = plt.subplots(1, 2, figsize=(13, 5))       # side-by-side subplots

axes[0].plot(np.arange(1, m+1), variance * 100, 'bo-', linewidth=1.2, markersize=8)  #
    variance per mode
axes[0].set_xlabel('Mode')
axes[0].set_ylabel('Variance Explained (%)')
axes[0].set_title('SVD Singular Value Spectrum')
axes[0].set_xticks(np.arange(1, m+1))                 # tick at each mode number
axes[0].grid(True, alpha=0.3)

for i, (col, label) in enumerate(zip(['b', 'r', 'g'], ['Mode 1', 'Mode 2', 'Mode 3'])):  #
    first 3 modes
    axes[1].plot(t, Y[i, :], col, linewidth=0.9, label=label)  # temporal evolution of each
        mode
axes[1].set_xlabel('Frame number')
axes[1].set_ylabel('Amplitude')
axes[1].set_title('SVD Modes (first 3)')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('fig_part3_svd.png', dpi=200, bbox_inches='tight')
```

```
plt.close()


#Figure 2: Cumulative variance

fig, ax = plt.subplots(figsize=(7, 5))                    # single subplot
cumvar = np.cumsum(variance) * 100                        # running total of variance
ax.plot(np.arange(1, m+1), cumvar, 'ro-', markersize=8, linewidth=1.2)  # cumulative curve
ax.axhline(y=95, color='k', linestyle='--', alpha=0.5, label='95% threshold')  # common
    threshold line
ax.set_xlabel('Number of Modes')
ax.set_ylabel('Cumulative Variance Explained (%)')
ax.set_title('Cumulative Variance - SVD')
ax.set_xticks(np.arange(1, m+1))
ax.set_ylim([0, 105]) # leave room above 100%
ax.legend()
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('fig_part3_cumulative_variance.png', dpi=200, bbox_inches='tight')
plt.close()


# Figure 4: Spatial structure (left singular vectors)

fig, ax = plt.subplots(figsize=(10, 5))                   # single bar chart
row_labels = ['Cam1 x', 'Cam1 y', 'Cam2 x', 'Cam2 y', 'Cam3 x', 'Cam3 y']  # channel names
x_pos = np.arange(m)      # bar positions
bar_w = 0.25  # bar width
for i in range(3):        # first 3 modes
    ax.bar(x_pos + i * bar_w, U[:, i], bar_w, label=f'Mode {i+1}', alpha=0.8)  # grouped bars
ax.set_xticks(x_pos + bar_w)                              # centre tick labels
ax.set_xticklabels(row_labels)
ax.set_ylabel('Weight (left singular vector)')
ax.set_title('Spatial Structure of SVD Modes')
ax.legend()
ax.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.savefig('fig_part3_mode_structure.png', dpi=200, bbox_inches='tight')
plt.close()
```

6.4 Supplementary Figures

```python
# extra figures not made in Part 1, 2 or 3

import numpy as np  # numpy for arrays and math
import scipy.io as sio  # to load .mat files
import matplotlib.pyplot as plt  # plotting
import matplotlib.patches as patches  # for drawing rectangles

# import functions and constants from part 1
from Part1Final import (rgb_to_brightness, apply_threshold,
                        morphological_cleanup, ROIS, THRESHOLDS,
                        MIN_BLOB_SIZE, MAX_BLOB_SIZE, VID_KEYS)


def fig_pipeline_cam1():
    # shows each step of tracking on cam1 frame 0
    cam_id = 1  # using camera 1
    data = sio.loadmat(f'cam{cam_id}.mat')  # load the mat file
    vid = data[VID_KEYS[cam_id]]  # get the video array
    frame0 = vid[:, :, :, 0].astype(np.float64)  # first frame as float

    y1, y2, x1, x2 = ROIS[cam_id]  # get roi bounds for this cam
    threshold = THRESHOLDS[cam_id]  # get threshold value

    roi = frame0[y1:y2, x1:x2, :]  # crop to roi
    bright = rgb_to_brightness(roi)  # compute brightness channel
    binary = apply_threshold(bright, threshold)  # threshold it
    cleaned, blob_info = morphological_cleanup(binary, MIN_BLOB_SIZE, MAX_BLOB_SIZE)  # clean
        up small blobs

    cx, cy = None, None  # centroid coords
    if blob_info:  # if we found any blobs
        best = max(blob_info, key=lambda b: b[2])  # pick biggest blob
        cx, cy = best[0], best[1]  # store its centroid

    fig, axes = plt.subplots(2, 3, figsize=(14, 8))  # 2x3 subplot grid

    # (a) full frame with roi rectangle
    axes[0, 0].imshow(frame0.astype(np.uint8))  # show original frame
    rect = patches.Rectangle((x1, y1), x2 - x1, y2 - y1,
        linewidth=2, edgecolor='lime', facecolor='none')  # green rectangle for roi
    axes[0, 0].add_patch(rect)  # add it to plot
    axes[0, 0].set_title('(a) Original frame + ROI')  # title
    axes[0, 0].axis('off')  # no axes

    # (b) cropped roi
    axes[0, 1].imshow(roi.astype(np.uint8))  # show cropped region
    axes[0, 1].set_title('(b) ROI crop')  # title
    axes[0, 1].axis('off')  # no axes

    # (c) brightness image
    axes[0, 2].imshow(bright, cmap='gray', vmin=0, vmax=255)  # show brightness as grayscale
    axes[0, 2].set_title(r'(c) Brightness $\max(R,G,B)$')  # title with formula
    axes[0, 2].axis('off')  # no axes

    # (d) histogram with threshold line
    axes[1, 0].hist(bright.ravel(), bins=100, color='steelblue', alpha=0.8)  # histogram of
        intensities
    axes[1, 0].axvline(threshold, color='red', linestyle='--',
                       linewidth=1.5, label=f'$\\theta = {threshold}$')  # vertical line at
                           threshold
    axes[1, 0].set_xlabel('Pixel intensity')  # x label
    axes[1, 0].set_ylabel('Count')  # y label
    axes[1, 0].set_title('(d) Intensity histogram + threshold')  # title
    axes[1, 0].legend()  # show legend

    # (e) binary mask after thresholding
    axes[1, 1].imshow(binary, cmap='gray')  # show binary image
    axes[1, 1].set_title('(e) Binary mask')  # title
    axes[1, 1].axis('off')  # no axes

    # (f) cleaned blob with centroid marked
    axes[1, 2].imshow(cleaned, cmap='gray')  # show cleaned mask
    if cx is not None:  # if we found a blob
        axes[1, 2].plot(cx, cy, 'r+', markersize=14, markeredgewidth=2)  # red cross at
```

```
        centroid
    axes[1, 2].set_title('(f) Cleaned blob + centroid')  # title
    axes[1, 2].axis('off')  # no axes

    plt.suptitle('Camera 1 - Image Processing Pipeline (Frame 0)',
                 fontsize=12, fontweight='bold')  # main title
    plt.tight_layout()  # fix spacing
    plt.savefig('fig_pipeline_cam1.png', dpi=200, bbox_inches='tight')  # save figure
    plt.close()  # close it


def fig_trajectory():
    # 2d trajectory scatter for each cam coloured by frame number
    data = np.load('tracking_results.npz')  # load tracking data

    fig, axes = plt.subplots(1, 3, figsize=(15, 5))  # 1 row 3 columns
    for i, cam in enumerate([1, 2, 3]):  # loop over cameras
        dx = data[f'cam{cam}_x']  # x positions
        dy = data[f'cam{cam}_y']  # y positions
        v = ~(np.isnan(dx) | np.isnan(dy))  # mask out nans
        sc = axes[i].scatter(dx[v], dy[v], c=np.arange(np.sum(v)),
                             cmap='viridis', s=4, alpha=0.7)  # scatter plot coloured by time
        axes[i].set_xlabel('Horizontal (px)')  # x label
        axes[i].set_ylabel('Vertical (px)')  # y label
        axes[i].set_title(f'Camera {cam}')  # title
        axes[i].grid(True, alpha=0.3)  # light grid
        axes[i].set_aspect('equal')  # equal aspect ratio
        plt.colorbar(sc, ax=axes[i], label='Frame', shrink=0.8)  # colorbar showing frame
            number

    plt.suptitle('2D Trajectory (colour = time)',
                 fontsize=13, fontweight='bold', y=1.02)  # main title
    plt.tight_layout()  # fix spacing
    plt.savefig('fig_trajectory.png', dpi=200, bbox_inches='tight')  # save
    plt.close()  # close


def _gabor_spectrogram(signal, width, t, n, tslide_step=2):
    # computes gabor spectrogram by sliding gaussian window and doing fft
    tslide = np.arange(0, n, tslide_step)  # time positions to slide window to
    Sgt_spec = np.zeros((len(tslide), n))  # empty spectrogram matrix
    for j, tau in enumerate(tslide):  # loop over each window position
        g = np.exp(-(t - tau)**2 / width**2)  # gaussian window centred at tau
        Sgt = np.fft.fft(g * signal)  # fft of windowed signal
        Sgt_spec[j, :] = np.abs(np.fft.fftshift(Sgt))  # store magnitude spectrum
    if Sgt_spec.max() > 0:  # normalise if not all zeros
        Sgt_spec /= Sgt_spec.max()  # divide by max
    return tslide, Sgt_spec  # return time positions and spectrogram


def fig_width_comparison():
    # spectrograms at 3 different gabor widths for horizontal and vertical
    data = np.load('tracking_results.npz')  # load data
    cam = 1  # camera 1
    dx = data[f'cam{cam}_x'] - np.mean(data[f'cam{cam}_x'])  # mean centred horizontal
    dy = data[f'cam{cam}_y'] - np.mean(data[f'cam{cam}_y'])  # mean centred vertical

    n = len(dx)  # number of frames
    t = np.arange(n)  # time vector
    k = np.arange(-n//2, n//2) / n  # frequency vector
    max_freq = 0.15  # max freq to show
    widths = [5, 15, 40]  # 3 different window widths
    freq_mask = (k >= 0) & (k <= max_freq)  # only positive freqs up to max

    # compute spectrograms for all widths
    results_x = {}  # store horizontal results
    results_y = {}  # store vertical results
    for w in widths:  # loop over widths
        tslide, spec_x = _gabor_spectrogram(dx, w, t, n)  # horizontal spectrogram
        _, spec_y = _gabor_spectrogram(dy, w, t, n)  # vertical spectrogram
        results_x[w] = (tslide, spec_x)  # save
        results_y[w] = (tslide, spec_y)  # save

    # horizontal displacement figure
    fig, axes = plt.subplots(len(widths) + 1, 1, figsize=(13, 3 * (len(widths) + 1)))  #
        subplots
```

```
    axes[0].plot(t, dx, 'k', linewidth=0.7)  # plot the signal on top
    axes[0].set_ylabel('Disp. (px)')  # ylabel
    axes[0].set_title(f'Camera {cam} - Horizontal displacement')  # title
    axes[0].set_xlim([0, n])  # set x limits
    for i, w in enumerate(widths):  # loop through widths
        tslide, spec = results_x[w]  # get this width result
        axes[i+1].pcolormesh(tslide, k[freq_mask], spec[:, freq_mask].T,
                             shading='auto', cmap='jet')  # plot spectrogram
        axes[i+1].set_ylabel('Freq\n(cyc/frame)')  # ylabel
        axes[i+1].set_title(f'Gabor width a = {w} frames')  # title
    axes[-1].set_xlabel('Frame number')  # xlabel on last subplot
    plt.tight_layout()  # fix spacing
    plt.savefig('fig_part2_width_comparison_horiz.png', dpi=200, bbox_inches='tight')  # save
    plt.close()  # close

    # vertical displacement figure
    fig, axes = plt.subplots(len(widths) + 1, 1, figsize=(13, 3 * (len(widths) + 1)))  #
        subplots
    axes[0].plot(t, dy, 'k', linewidth=0.7)  # plot signal
    axes[0].set_ylabel('Disp. (px)')  # ylabel
    axes[0].set_title(f'Camera {cam} - Vertical displacement')  # title
    axes[0].set_xlim([0, n])  # x limits
    for i, w in enumerate(widths):  # loop through widths
        tslide, spec = results_y[w]  # get result
        axes[i+1].pcolormesh(tslide, k[freq_mask], spec[:, freq_mask].T,
                             shading='auto', cmap='jet')  # plot spectrogram
        axes[i+1].set_ylabel('Freq\n(cyc/frame)')  # ylabel
        axes[i+1].set_title(f'Gabor width a = {w} frames')  # title
    axes[-1].set_xlabel('Frame number')  # xlabel
    plt.tight_layout()  # fix spacing
    plt.savefig('fig_part2_width_comparison_vert.png', dpi=200, bbox_inches='tight')  # save
    plt.close()  # close


def fig_svd_reconstruction():
    # shows rank 1 vs rank 2 reconstruction of cam1 signals
    data = np.load('tracking_results.npz')  # load tracking data

    cam1_x = data['cam1_x'];  cam1_y = data['cam1_y']  # cam 1 positions
    cam2_x = data['cam2_x'];  cam2_y = data['cam2_y']  # cam 2 positions
    cam3_x = data['cam3_x'];  cam3_y = data['cam3_y']  # cam 3 positions

    n = min(len(cam1_x), len(cam2_x), len(cam3_x))  # shortest length
    X = np.vstack([cam1_x[:n], cam1_y[:n],
                   cam2_x[:n], cam2_y[:n],
                   cam3_x[:n], cam3_y[:n]])  # stack all into matrix
    t = np.arange(n)  # time vector

    mn = np.mean(X, axis=1, keepdims=True)  # mean of each row
    B = X - mn  # subtract mean
    U, S, Vt = np.linalg.svd(B / np.sqrt(n - 1), full_matrices=False)  # do svd

    fig, axes = plt.subplots(2, 2, figsize=(13, 8))  # 2x2 grid

    for rank, ax_col in zip([1, 2], [0, 1]):  # rank 1 and rank 2
        X_approx = U[:, :rank] @ np.diag(S[:rank]) @ Vt[:rank, :] * np.sqrt(n - 1) + mn  #
            reconstruct

        # horizontal comparison
        axes[0, ax_col].plot(t, X[0, :], 'k', linewidth=0.7, label='Original')  # original
            signal
        axes[0, ax_col].plot(t, X_approx[0, :], 'b--', linewidth=0.9, label=f'Rank-{rank}
            approx')  # approximation
        axes[0, ax_col].set_title(f'Cam1 horizontal - rank {rank}')  # title
        axes[0, ax_col].set_ylabel('Position (px)')  # ylabel
        axes[0, ax_col].legend(fontsize=9)  # legend
        axes[0, ax_col].grid(True, alpha=0.3)  # grid

        # vertical comparison
        axes[1, ax_col].plot(t, X[1, :], 'k', linewidth=0.7, label='Original')  # original
        axes[1, ax_col].plot(t, X_approx[1, :], 'r--', linewidth=0.9, label=f'Rank-{rank}
            approx')  # approx
        axes[1, ax_col].set_title(f'Cam1 vertical - rank {rank}')  # title
        axes[1, ax_col].set_xlabel('Frame number')  # xlabel
        axes[1, ax_col].set_ylabel('Position (px)')  # ylabel
        axes[1, ax_col].legend(fontsize=9)  # legend
```

```
        axes[1, ax_col].grid(True, alpha=0.3)  # grid

    plt.tight_layout()  # fix layout
    plt.savefig('fig_part3_reconstruction.png', dpi=200, bbox_inches='tight')  # save figure
    plt.close()  # close


if __name__ == '__main__':  # run all figures
    fig_pipeline_cam1()  # pipeline figure
    fig_trajectory()  # trajectory scatter
    fig_width_comparison()  # width comparison spectrograms
    fig_svd_reconstruction()  # svd reconstruction
```

6.5 Additional Figures

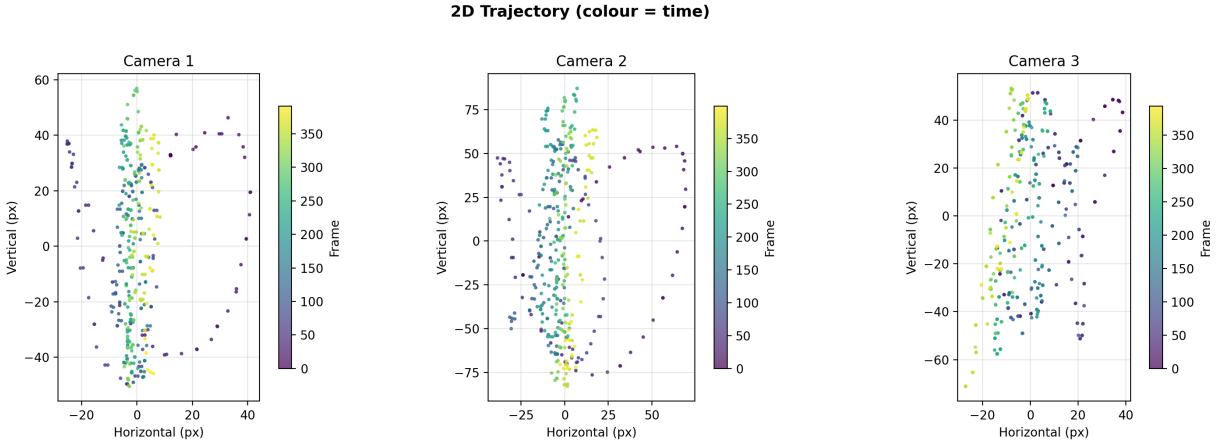


Figure 6: 2D trajectory of the oscillating mass from each camera, coloured by frame number.

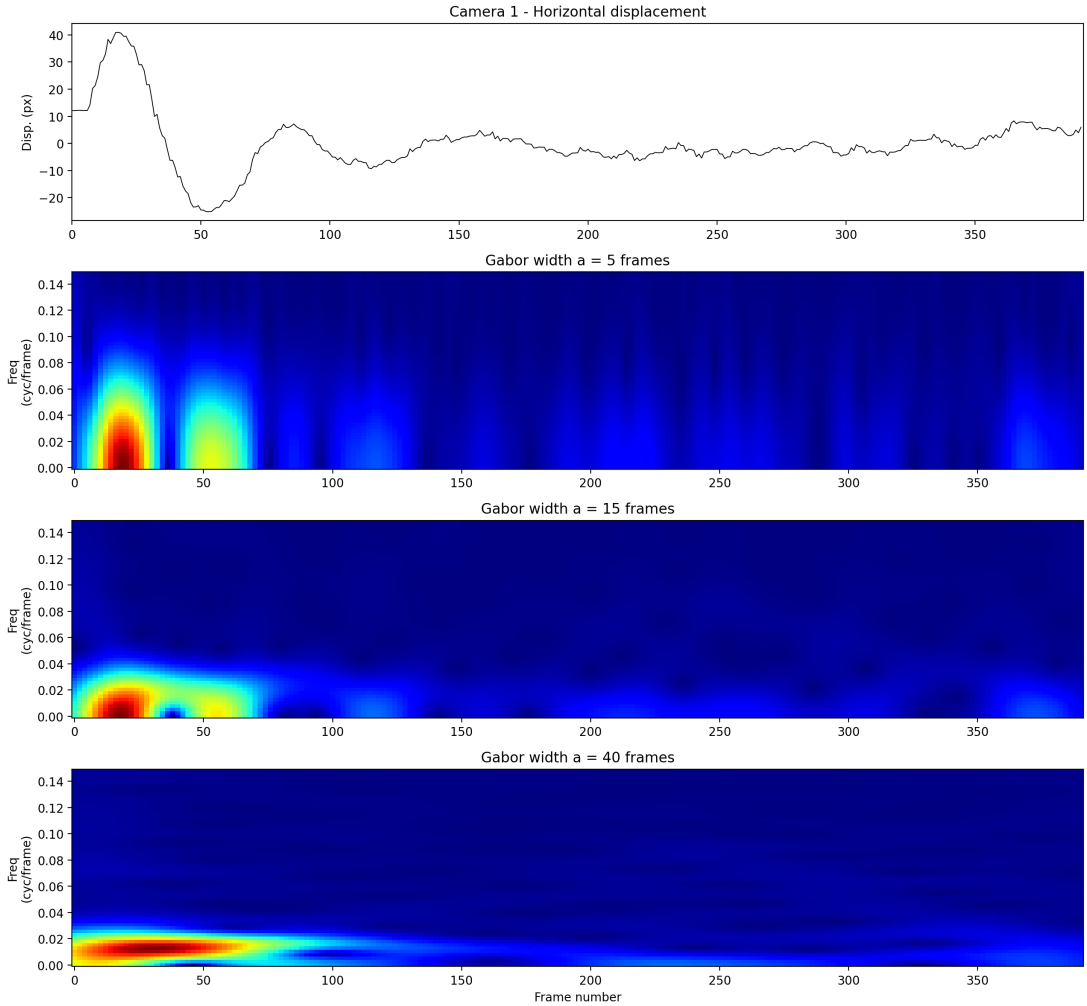


Figure 7: Horizontal displacement spectrograms at three Gabor widths ($a = 5, 15, 40$), showing the time-frequency resolution trade off.

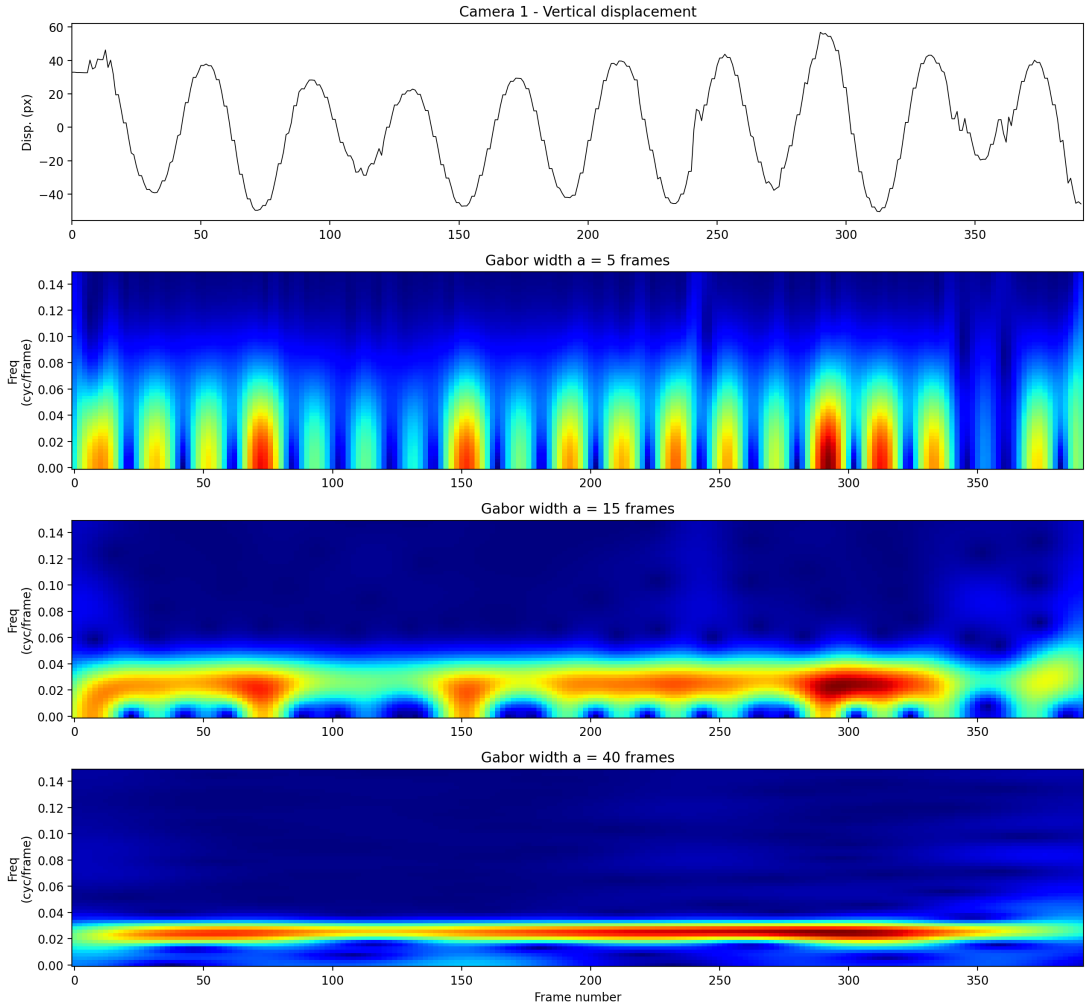


Figure 8: Vertical displacement spectrograms at three Gabor widths ($a = 5, 15, 40$).

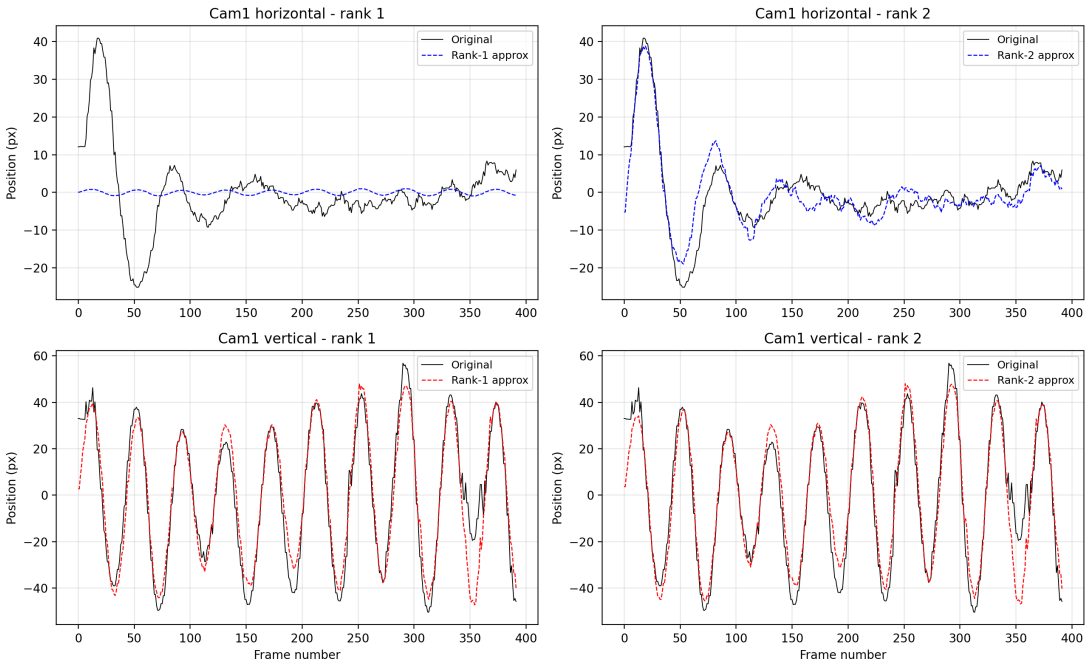


Figure 9: Low rank reconstruction of Camera 1 signals: rank 1 (left) vs rank 2 (right) approximation.

References

Breitenberger, E. and Mueller, R.D. (1981) 'The elastic pendulum: A nonlinear paradigm', *Journal of Mathematical Physics*, 22(6), pp. 1196-1210.

Gabor, D. (1946) 'Theory of communication', *Journal of the Institution of Electrical Engineers*, 93(26), pp. 429-457.

Gonzalez, R.C. and Woods, R.E. (2018) *Digital Image Processing*. 4th edn. New York: Pearson.

Hartley, R. and Zisserman, A. (2003) *Multiple View Geometry in Computer Vision*. 2nd edn. Cambridge: Cambridge University Press.

Kuglin, C.D. and Hines, D.C. (1975) 'The phase correlation image alignment method', *Proceedings of the IEEE International Conference on Cybernetics and Society*, pp. 163-165.

Kutz, J.N. (2013) *Data-Driven Modeling and Scientific Computation*. Oxford: Oxford University Press.

Lynch, P. (2002) 'Resonant motions of the three-dimensional elastic pendulum', *International Journal of Non-Linear Mechanics*, 37(2), pp. 345-367.

Nayfeh, A.H. and Mook, D.T. (1979) *Nonlinear Oscillations*. New York: John Wiley & Sons.

Turk, M. and Pentland, A. (1991) 'Eigenfaces for recognition', *Journal of Cognitive Neuroscience*, 3(1), pp. 71-86.